Review 2 implementation learning guide

For Aarya, Shashannk, Rohan and Tusti | 5 November 2026

1. The idea and where we are

We simulate a team collecting food and bringing it to a nest. Each agent sees only its own nearby cells. Agents communicate indirectly by changing food and home pheromone fields shared by the environment. This is stigmergy: one agent changes the environment, and another can respond later. There are no messages, trail author IDs or individual-agent trust scores.

The research question is whether a lightweight defense using local pheromone history and the agent's own progress can improve learned cooperative retrieval under bounded false-food injection, including unseen conditions, without a large clean-performance cost. Simulation results do not establish physical-robot or real-world performance.

Stage	What exists	What it establishes
Review 1	Grid simulator, shared PPO, persistent injector, local history and timeout comparison	Executable mechanics/pipeline; old weak-policy and harmful-timeout results are preserved
Phase 2A	Fair separately trained baselines, candidate calibration, local diagnostics	Candidate J passed a reported clean stochastic development gate
Phase 2B	Three persistent-attack calibrations, separated corpus and agent-step CSV exporter	Reproducible development data engineering; no consistent attack degradation
Future	Validated exposure labels, final grouped splits/windows, learned detector/mitigation and held-out evaluation	Not completed; no learned-defense or robustness claim

Phase 2A/2B are internal work packages. They are different from academic Phase-I/Phase-II. Review 2 presents both the positive clean calibration and the negative attack calibration.

Reading map

Contents: 2 Grid | 3 Local view | 4 PPO | 5 Clean gate | 6 Limits | 7 Attacks | 8 Logs | 9 Corpus/CSV | 10 Files | 11 Commands | 12 Review answers

Evidence boundary

Original Candidate J checkpoints and the full generated corpus are absent from this checkout. Their numerical results are previously recorded in decisions 0012-0016 and the changelog, not independently rerun here. This audit executes focused/full tests, a small training smoke and a tiny real rollout-to-CSV integration. Use the audit record for exact fresh check counts. Source revisions are printed in this PDF's footer metadata.

Sources: AGENTS.md; README.md; docs/CONTEXT.md; docs/decisions/0012-phase2a-scout-follow-capacity.md; docs/decisions/0016-phase2b-development-calibration-corpus.md.

Grid mechanics and pheromone values

2. A complete simulator step

Reset initializes a seeded square map, nest, food quantities, agent positions, empty pheromone fields, event flags and nest-visit counters. Attacker selection uses a separate seeded random stream, so selecting an attacker does not consume the map/contention generator. Coordinates are (row, column), starting at zero in the top-left; row increases downward.

Every world step validates one action per live agent before changing state. Agents move simultaneously in the sense that movement does not exclude occupied cells: co-location is allowed. Out-of-grid movement is blocked. Successful movement outside the nest increments a per-agent counter; occupying a nest cell resets it, including carrying visits. Pickup/delivery is then processed in a seeded random agent order, resolving competition for the last food unit.

A carrying agent in the nest delivers one unit and becomes empty. Otherwise an empty agent on nonempty food picks up one unit. The resulting carrying state chooses the honest deposit channel. Next comes any false-food injection, common clipping, evaporation, updated observations, shared reward and terminal/truncation flags. No diffusion or obstacles are implemented.

Deposits and evaporation

For deposit amount d, nest-visit movement counter c and home decay q = 0.95:

Agent after pickup/delivery	Deposit at its current occupied cell
Carrying	Food pheromone += d, including waiting
Empty and inside nest	Home pheromone += d
Empty, successfully moved outside nest	Home pheromone += d * q^c
Empty, stationary or blocked outside nest	No home deposit

For each field cell, P_next = (1 - e) * clip(P_current + sum(honest deposits) + applied injection, 0, C). Candidate J uses d = 1, e = 0.02 and C = 10. Several agents can contribute at the same cell. Both honest and injected food pheromone follow the same cap and evaporation.

Worked example

An empty agent's first movement outside the nest gives c = 1 and deposit 0.95. With an initially zero cell and 2% evaporation, the displayed value becomes 0.95 * 0.98 = 0.931. Its second outside movement deposits 0.9025 at the new cell. If it waits there next, the old 0.88445 concentration simply becomes 0.866761; it adds no home pheromone. Historical Review 1 screenshots used 10% evaporation: that multiplies by 0.9, rather than subtracting a fixed 0.1.

Food and home values are concentrations, not food counts or distances. Recency and overlapping trails can make a value high near the nest or elsewhere; the rule does not guarantee a perfect gradient. The conservation check is remaining + carried + delivered = initial food. For Candidate J, the initial total is 100.

Sources: src/stigmergy/environment.py; docs/decisions/0001-clean-grid-mechanics.md; docs/decisions/0005-nest-anchored-home-review.md.

What one local agent can see

3. The 58-value observation

A 3 x 3 crop contains nine cells, each with six channels. It is flattened in row-major order: 9 * 6 = 54 entries. Four own-state flags are appended. The PPO actor gets this vector at action time directly from the simulator; reading a saved log is not required to choose the next action.

Channel in each cell	Meaning and scaling
0	Food present: 0 or 1, not the global food quantity
1	Nest present: 0 or 1
2	Food pheromone / field_cap
3	Home pheromone / field_cap
4	Number of agents occupying the cell / total agents
5	Outside-grid boundary indicator
Final entries 54-57	Carrying, just picked up, just delivered, just blocked

The center is patch [1,1]. Off-grid cells set only their boundary bit. Inputs exclude absolute coordinates, global maps, attacker IDs/timing, privileged labels and trail authors.

Actions and an example

Action 0 stays; 1 moves up; 2 right; 3 down; 4 left. From (row 1, column 2), action 2 attempts (1,3). At the right border it is blocked instead. A food patch outside the crop is invisible. Agents at different locations receive different vectors even though they use the same network weights.

Twelve local-history features

Features	Interpretation
Food mean, food maximum	Statistics over the local food-pheromone crop
Home mean, home maximum	Statistics over the local home-pheromone crop
Food mean delta, home mean delta	Current local mean minus the previous local mean
Carrying, pickup, delivery, blocked	Own state and recent local task events
Steps without progress	Steps without pickup/delivery, capped after division by progress_horizon (default 20)
Revisit fraction	(Observations seen - distinct inferred cells) / observations seen

One extractor exists per agent and resets per episode. Relative displacement is reconstructed from the preceding executed action and the current blocked flag. The initial observation is not counted as a completed unproductive step. Means/maxima and flags are in [0,1]; mean deltas can be negative, within [-1,1]. These 12 features are logged for the local-defense foundation. They are not appended to the current 58-input PPO network, and there is no fitted attack detector yet.

Sources: src/stigmergy/environment.py; src/stigmergy/defense.py; src/stigmergy/observations.py; tests/test_defense.py; tests/test_observations.py.

Shared PPO: training and action choice

4. What is trained?

One parameter-shared neural policy is applied separately to each agent's 58-value vector. The actor and local value network each use two 32-unit layers. There is no centralized global-state critic or identity embedding. All agents receive the same reward: the number of food units delivered by the team on that world step. Pickup alone does not provide an extra shaped reward.

The simulator creates observations and rewards. PPO collects observation/action/reward/next-observation transitions, estimates returns and advantages using the local value predictions, then updates the policy and value network. Logs are outputs for reproducibility and analysis; the current PPO does not train by loading the exported corpus CSV. A later supervised detector would use a separately prepared labelled dataset.

Adapter and training cycle



Rollout collection -> advantage/value estimates -> PPO updates -> checkpoint

SharedGridVecEnv exposes one slot per agent and batches observations as (N,58), actions/rewards/done flags as (N,). The slots interact in one grid; they are not independent worlds. All reset together. At an episode boundary the adapter returns the next map's initial observations and preserves the previous terminal observation in info. TimeLimit.truncated marks a horizon cutoff so SB3 can bootstrap appropriately.

Maps cycle through train seeds 0-15; diagnostic maps 100-103 remain development data. The standard settings are rollout n_steps = 128 per slot, minibatch 64, four optimization epochs, learning rate 0.0003 and gamma = 0.99. Requested training budget is 300,000 agent transitions per job. The library rounds to full rollout batches: with four agents, one batch is 512 transitions. Actual counts belong in each run summary.

Deterministic versus stochastic

The policy outputs action probabilities. With probabilities right 0.40, down 0.35 and stay 0.25, deterministic evaluation chooses right. Stochastic evaluation samples according to those probabilities; it can choose down or stay. A recorded action seed makes this sampling repeatable on the same stack. These are two ways to execute the same checkpoint, not two separately trained algorithms. A deterministic policy can loop even when stochastic actions occasionally escape.

Training saves initial.zip, policy.zip, progress.csv, a provenance manifest and before/after diagnostic summaries. Finite parameters, changed weights and save/load predictions are checked. Clean/no-pheromone jobs start separately; evaluation keeps the checkpoint frozen. Small smoke results establish pipeline execution, not useful learning or robustness.

Sources: src/stigmergy/training.py; src/stigmergy/ppo_adapter.py; configs/phase2a/policy/full_seed_7.json; tests/test_training.py.

Phase 2A: clean coordination checks

5. Fair comparisons before attack data

An attack on pheromone is hard to interpret if the learned policy gains nothing from pheromone in the first place. Phase 2A therefore compares full-input PPO, a separately trained no-pheromone PPO, and random actions on the same development task. Same-seed PPO pairs retain identical architecture, optimizer, learning budget and train/diagnostic maps. Only observation_mode differs. The audit now enforces that equality rather than relying on matching configuration files.

The no-pheromone baseline keeps all 58 input positions but sets exactly 18 copied food/home entries to zero during both training and evaluation. Other channels remain. It does not erase the simulator's field or change local trajectory records. Review 1's frozen-policy input ablation masked an already trained model only at evaluation; that is a sensitivity test, not this fair trained baseline.

```
# Nine cells; channels 2 and 3 are the pheromone entries.
indices = tuple(cell * 6 + ch for cell in range(9) for ch in (2, 3))
transformed = observations.copy()  # Simulator observations stay intact.
transformed[..., indices] = 0     # Only in the no_pheromone mode.
```

Candidate J and the reported clean gate

The task is a 12 x 12 grid, four agents, a top-left 2 x 2 nest, two distant adjacent food cells with 50 units each, horizon 1,100 and 2% evaporation. Cluster placement varies by map seed. Three training seeds (7,17,27) each have fresh full and no-pheromone models: six PPO jobs, plus three seeded random references. Diagnostic means cover four maps.

Training seed	Full PPO	No-pheromone PPO	Random	Full minus no-pheromone
7	100.00	68.75	3.00	+31.25
17	80.00	63.50	1.75	+16.50
27	100.00	80.50	2.25	+19.50

These are previously recorded stochastic mean team deliveries, not new audit measurements. The predeclared gate requires full input to strictly beat both references for every training seed. J passes that development gate; it is not a statistical-significance test, held-out result or robustness claim. For example, seed 17's paired advantage is 80.00 - 63.50 = 16.50 food units. Training-seed variability and evaluation-episode variability are distinct.

The audit preserves old runs and adds input/source/artifact hashes to new calibration roots. Resume rejects modified evidence before updating a manifest. Legacy roots without the necessary hashes must use fresh output, rather than silently mixing experiments.

Sources: src/stigmergy/calibration.py; src/stigmergy/observations.py; docs/decisions/0006-phase2a-calibration-contract.md; docs/decisions/0012-phase2a-scout-follow-capacity.md; docs/decisions/0017-review2-provenance-validation.md.

Failed candidates and limitations

6. Why J was investigated

Candidates	Hypothesis or change	Previously recorded finding
A-C	Larger or denser eight-agent development maps	Mixed full/no-pheromone differences; each failed the strict three-seed gate
D-E	Smaller search space and repeat retrieval	Better than random, but ties/losses against no-pheromone remained
F	Change E evaporation from 10% to 2%	More visible food trail; seed 27 still lost the baseline comparison
G	Four-agent distant adjacent cluster, 30 units per cell and long horizon	Closest clean result: differences 0.00/+8.75/+5.75; seed 7 tied at 60-unit capacity
H	Shorten G's horizon to 700 steps	Full-input learning became unstable
I	Double training on H to 600,000 transitions	Did not resolve instability
J	Retain G's task; increase only capacity to 50 per cell	Passed the reported stochastic three-seed delivery gate

A tie at the maximum can hide a speed difference: two policies can both retrieve all 60 units, with one finishing sooner. The written gate still required a strict delivery win, so G did not pass. J tested the capacity-ceiling explanation while retaining the stable long horizon, rather than changing several parameters together. Failed candidates must stay visible; J was developed on these maps, so they cannot become final unseen test maps.

What the diagnostics actually measure

The exposure utility measures whether food/home pheromone is locally visible. The action-sensitivity utility compares a frozen model's action distribution on the same real observation with and without its local pheromone entries. Candidate G's reported mean visible-observation total variation was 0.272/0.187/0.230 for seeds 7/17/27. This shows sensitivity, not beneficial communication, harmful exposure or detector accuracy.

J's deterministic full-policy means were 100.00, 0.00 and 60.00. Thus the positive clean finding concerns stochastic execution. We cannot say the learned coordination is uniformly reliable in every action mode or seed.

How to explain the result in a review

“We first checked whether pheromone actually helps the learned policy under fair training. Several candidates failed. Candidate J passed our predeclared stochastic development gate across three training seeds, but deterministic behavior remains mixed. We then froze the clean policies and tested bounded injection. This is clean calibration evidence, not proof of a successful defense.”

The review expectation for a wider 30+ relevant-paper survey remains open. Software documentation and dataset resources are not research papers, and the existing reference list must not be described as completing that target.

Sources: docs/PHASE2A_CALIBRATION.md; docs/decisions/0006-0012; src/stigmergy/exposure.py; src/stigmergy/action_sensitivity.py.

Phase 2B: attacks and matched controls

7. What the attacker can change

One agent out of four is selected at reset and remains selected for that episode: 25%, not the earlier eight-agent review fixture's 12.5%. It keeps productive PPO movement and may add nonnegative false-food pheromone only at its actual post-movement occupied cell. It cannot create food, erase trails, write anywhere on the map, change rewards or modify the policy. Shared fields contain no author identity, so following a trail cannot update an individual agent's reputation.

Request/step cap is 1 unit. Authorization is the minimum of request and remaining per-step/episode applied-mass budgets. Applied mass also respects available food-field headroom. Example: request = 1, remaining budget = 0.7 and headroom = 0.2 gives authorized = 0.7 but applied = 0.2. Budgets consume applied mass. The cap can prevent part of a request, so requested, authorized and applied totals differ.

Scenario	What happens	Purpose
Clean	No injector; all agents are treated as honest	Normal clean-task reference
Injection-disabled	Same seeded selected-agent designation as attacked, zero injection	Matched control for the injection treatment
Attacked	Same designation and productive policy, bounded false-food deposits	Measure injection effects

Each matrix freezes three Candidate J full-input checkpoints, uses four diagnostic maps and stochastic action seed 7: 12 groups x 3 scenarios = 36 episodes. Local clean/disabled records must match exactly; disabled/attacked selection must match. Counts for honest delivery exclude the selected agent in those two scenarios. Compare their identical honest population, rather than honest clean versus honest attacked.

Previously recorded negative results

Persistent calibration	First allowed deposit	Episode cap	Mean honest injection loss
Immediate	Transition 1	20 units	-2.17
Longer immediate	Transition 1	200 units	-0.42
Delayed	Transition 201	200 units	-2.25

Injection loss = honest delivery(disabled) - honest delivery(attacked). A loss of -2 means attacked delivered two more honest units in that pair, not two fewer. All three means are nonpositive and group effects were mixed. They do not establish consistent degradation, general harmlessness, a beneficial attack or successful defense. The delayed mode leaves transitions 1-200 untouched; injected pheromone may linger after active deposits end.

No detector is fitted. The earlier timeout rule remains a local baseline: after eight completed unproductive steps it can redirect toward a visible adjacent goal or cycle legal movement. Its triggers are not attack labels. No learned-defense trajectories are part of this three-scenario corpus. Decoy/intermittent modes require a new documented calibration decision and remain unimplemented.

Sources: src/stigmergy/attacks.py; src/stigmergy/phase2b.py; src/stigmergy/evaluation.py; docs/decisions/0013-0015.

What gets logged, and when

8. One transition record

At time t, the controller receives O_t, updates each agent's history using its preceding executed action, and computes features H_t. PPO proposes A_t; a timeout run may override it to A'_t. The simulator executes A'_t and returns O_(t+1), reward and outcomes. The writer then appends one JSONL record numbered step = t+1. Thus step 1 contains the initial observation before the first transition and the outcome after it.

Location/field	Contents	Can it be a current detector input?
local.observations_before	Per-agent 58-value O_t	Candidate local input
local.features	Per-agent 12-value H_t	Candidate local-history input
proposed_actions / executed_actions	Proposal and actual executed action	Logged; not default CSV inputs
observations_after / rewards / infos	Next observation, shared reward, own pickup/delivery/blocked outcomes	Outcomes; avoid using future information to predict the current decision
terminations / truncations / timeout_active	Episode boundary and baseline activity flags	Logged outcomes/diagnostics, not detector labels
simulator_metadata	Attack events, total/honest delivery	Audit only; never model features

```
step: 1
local:
  observations_before: {agent_0: [58 local values], ...}
  features:            {agent_0: [12 history values], ...}
  proposed_actions:    {agent_0: 2, ...}
  executed_actions:    {agent_0: 2, ...}
  ... next observations, reward and typed outcomes ...
simulator_metadata:
  attack_events:       [... requested/authorized/applied mass ...]
  total_delivered:     ...
  honest_delivered:    ...
```

This is a structural illustration, not a fabricated measured episode. Agent keys route rows and per-agent history; they are not numeric identity inputs to PPO. Global arrays and attacker selection belong only to simulator-side writers.

Files around the transition log

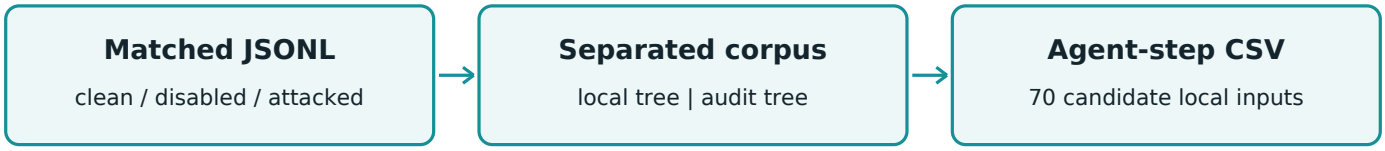
manifest.json records purpose, completion status, configuration, source hashes, seeds and policy provenance. Each episode.json stores episode metrics and selected-agent audit accounting. episodes.csv aggregates episode rows; contrasts.csv contains paired differences; summary.json contains aggregate/per-seed results. group_splits.json preserves scenario families together. A development assignment is not a final train/validation/test split.

Training separately saves progress.csv, initial/final model ZIPs, version information and before/after diagnostics. Human snapshots and plots are generated artifacts. Applied injection mass is ledger accounting, not an exposure label; local field visibility is also not a harm label. Full historical raw artifacts are unavailable here, so this audit cannot freshly confirm every reported row or figure.

Sources: src/stigmergy/evaluation.py; src/stigmergy/trajectories.py; src/stigmergy/training.py; src/stigmergy/phase2b.py.

Calibration corpus and agent-step CSV

9. Corpus and export boundaries



Audit only -> future validated labels; grouped splits -> windows -> detector (planned)

The reported corpus has 3 settings x 3 policies x 4 maps = 36 development groups, each with three scenarios: 108 episodes. Calibration-prefixed group IDs avoid collisions. Related maps remain development data; these groups do not authorize random row splits.

model_observable/ contains local steps; audit/ contains attack/global accounting. The exporter reads the root audit_index.jsonl for provenance and paths, but never opens privileged audit step files. Paths and scenario names are not features.

The agent-step CSV

The reported 103,004 world transitions x 4 agents give 412,016 rows; episode lengths vary. These are historical totals. Only a tiny integration corpus is freshly rebuilt in this audit.

Column family	Count	Allowed use
Provenance	10	Calibration/group/scenario/split, seeds, mode, agent ID and step; grouping/audit only
Local observation inputs	58	observation_before_00 through observation_before_57
History inputs	12	history_ + each named feature
Local outcomes	9	Proposal, executed action, reward, terminal/truncated, blocked/pickup/delivery and timeout activity

schema.json marks 70 of 89 columns as candidate inputs. Do not feed all columns to a classifier. H_t uses O_t and earlier history; reward/pickup after A_t is a later outcome, not input at t.

What the audit fixes

Corpus and exporter now reject incomplete or duplicate scenario groups, conflicting map/seed/split provenance, unknown local fields, wrong shapes, non-finite values, invalid actions and non-Boolean outcomes. Repeated valid builds/exports are checked for reproducibility; existing output directories are not overwritten. Failures and interrupts retain explicit failed/incomplete status.

There are no labels, windows or final partitions. Future labels must track residual injected contribution through evaporation/clipping, not just deposit times or attacker IDs. Validate attribution, group maps/episodes before windows, fit on training groups, tune on validation and reserve unseen maps/attacks.

Sources: src/stigmergy/phase2b_corpus.py; src/stigmergy/phase2b_dataset.py; docs/PHASE2B_DATASET_ROADMAP.md; CHANGELOG.md (2026-11-02).

Files and recommended reading order

10. Start with the contract, then follow execution

Read AGENTS.md, README.md, docs/CONTEXT.md and the relevant decision records first. They distinguish implemented facts, historical outcomes and planned research. Start reading code with environment.py, then observations.py, ppo_adapter.py/training.py, calibration.py, attacks.py/evaluation.py and the three Phase 2B modules.

Source module	Main responsibility, state and outputs
environment.py	GridConfig / ResourceRetrievalEnv; world state, reset/step and local outputs.
observations.py	Copy local input batches; zero only the 18 pheromone entries when declared.
ppo_adapter.py	SharedGridVecEnv; coupled slots, map cursor, auto-reset and terminal views.
training.py	TrainingConfig, PPO updates, diagnostics, hashes and checkpoint reload.
calibration.py	Candidate/seed/mode runs, fair pairs, random reference, gate and safe resume.
policies.py	Local rule-based debugging controller; not learned-policy evidence.
attacks.py	AttackConfig, selected subset, onset, occupied-cell deposits and mass ledger.
defense.py	Per-agent local-history state and fixed timeout baseline; no classifier.
evaluation.py	Frozen controller, matched rollouts, local/audit logs and delivery contrasts.
evaluation_plots.py	Plots recorded comparisons; does not change experiment outcomes.
trajectories.py	Fixture recorder, JSON conversion, provenance and matched attack configs.
phase2b.py	Frozen-checkpoint checks, calibration triplets, group manifests and reports.
phase2b_corpus.py	Validate source groups/local records; write separated archive trees.
phase2b_dataset.py	Validate corpus families; export local CSV, schema and manifests.
exposure.py / action_sensitivity.py	Local visibility and frozen-input sensitivity diagnostics; not a detector.
cli.py / __init__.py	Command routing and exports; training dependencies are optional imports.

Other files

Environment/policy JSON defines grids, maps, budgets and input modes; matrix and attack/corpus plans define runs. JSON comments are invalid: document fields in linked guides. Never edit a plan and resume old output.

Tests mirror mechanics, locality, training, pairing and data contracts. Requirements locks pin dependencies; pyproject.toml declares the package. egg-info is generated installation metadata. scripts/ contains evidence/diagnostic wrappers and the PDF renderer. Decisions define protocols; context/changelog record verification. Checkpoints, JSONL/CSV and plots belong in ignored artifacts/ or data/generated/, not bulk Git commits.

Sources: repository source/config/test tree; README.md; docs/REVIEW2_AUDIT.md.

Setup, commands and artifact inspection

11. Use the verified runtime

Use Python 3.12 for the complete pinned training stack. Python 3.11 remains advertised for core use, but requirements-training.lock contains contourpy 1.4.0, which requires Python >=3.12. This audit used Linux Python 3.12.14 and the unchanged lock. Cross-platform bitwise identity is not promised.

```
# Linux: run from the selected branch checkout.
python3.12 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements-training.lock
python -m pip install -e . --no-deps
python -m pip check
python -m pytest -q

# Windows PowerShell: same commands after this activation.
py -3.12 -m venv .venv
.\.venv\Scripts\Activate.ps1
```

Small checks versus full experiments

```
python -m stigmergy.cli demo --config configs/env/development.json --seed 7 --output \
  artifacts/demo-new
python -m stigmergy.cli debug-policy --output artifacts/debug-new
python -m stigmergy.cli record-fixture --help
python -m stigmergy.cli train --policy-config configs/policy/review-smoke.json --output \
  artifacts/smoke-new
python -m stigmergy.cli phase2a-calibrate --plan \
  configs/phase2a/calibration-scout-follow-capacity.json --output artifacts/phase2a/capacity-new
```

The last command is the full six-model Candidate J experiment, not a quick smoke. Do not run it just to view the PDF. Use --help to inspect command options. A --resume invocation uses the same plan/output and now requires bound inputs and unchanged completed child evidence. Historical roots without hashes need a fresh output rather than overwriting history.

Phase 2B depends on original artifacts

```
python -m stigmergy.cli phase2b-calibrate --plan configs/phase2b/attack-calibration.json --output \
  artifacts/phase2b/calibration-new
python -m stigmergy.cli phase2b-build-corpus --plan \
  configs/phase2b/development-calibration-corpus.json --output data/generated/corpus-new
python -m stigmergy.cli phase2b-export-agent-steps --corpus data/generated/corpus-new --output \
  data/generated/csv-new
```

These commands need the plan's exact completed checkpoints/source runs. Git does not include ignored artifacts, so a fresh clone alone is insufficient. Restore a verified artifact bundle at its declared paths; check hashes and completed manifests first. Do not replace Candidate J with a newly trained unrelated model or silently edit an old run. The two 200-unit calibration plans are separate versioned commands, not parameters to tune after viewing results.

All output directories must be new. Inspect manifest.status before reading a result: started/failed/incomplete is not completed. Check configuration, seeds, source/parameter hashes, group counts and schema input columns. A missing checkpoint is a dependency error, not proof of broken learning. The fresh audit smoke and tiny integration remain distinct from the historical J/108-episode evidence.

Sources: README.md; requirements-training.lock; src/stigmergy/cli.py; docs/REVIEW2_AUDIT.md.

Review explanations and remaining work

12. Short answers you can use

Likely question	Accurate answer
What changed since Review 1?	Fair trained baselines, clean calibration, frozen attack matrices, separated corpus/CSV and stronger validation.
Why are some attack results equal or better?	Injection changed fields but did not consistently reduce delivery; the cause remains unresolved.
Is timeout our learned defense?	No: a fixed local baseline, previously harmful. Learned detection/mitigation is pending.
Do simulator labels drive PPO?	No: PPO uses local observations and rewards. Privileged metadata is audit-only; exposure labels are pending.
Is the dataset ready?	The development corpus/CSV exists; labels, final splits and windows do not.
Can we claim robustness?	No: clean development benefit and negative attack calibration are not held-out defense evidence.

Present the two main implementation messages

“We established a clean stochastic pheromone benefit with fair three-seed training. We then froze the policies and tested matched bounded injection. None of three settings consistently reduced delivery. We preserved the negative evidence in a development corpus and inspectable CSV; learned defense remains pending.”

Keep the existing 22-slide Review 2 style and 6 November date. Retain J's chart and deterministic limitation; update Phase 2B implementation/data/timeline content and the recorded losses. Exact text/notes: docs/REVIEW2_SLIDE_UPDATES.md. Canva access was unavailable: prepared text is not a saved deck edit.

Remaining-work checklist

• Recover original artifacts to independently verify historical results.

• Declare a new bounded attack hypothesis before decoy/intermittent calibration; preserve negative results.

• Validate residual-exposure attribution and benign congestion/depletion/saturation controls.

• Declare grouped training/validation/final maps and withheld attacks before windows; fit preprocessing on training only.

• Train local logistic-regression/random-forest detectors, validate thresholds and integrate mitigation; collect learned-defense trajectories.

• Complete undefended, trained no-pheromone, timeout and cautionary baselines; test unseen conditions, clean cost and uncertainty.

• Extend literature coverage; complete reproducibility/release work. Publication remains conditional.

Evidence and sources

Fresh checks: docs/REVIEW2_AUDIT.md. Historical numbers: decisions 0012-0016 and the 2026-11-02 changelog. Validation contract: decision 0017. This guide distinguishes current implementation from future research; it does not certify roadmap completion.